Another Introduction to Guarded Transition Systems

Olivier Cornes

April 15, 2026

Abstract

This is a brief introduction to guarded transition systems (GTS) for safety practitioners. GTS are introduced by illustrating the need for a powerful formalism for the analysis of the reliability of complex systems and how this integrates into system safety engineering. GTS is compared to series of other reliability engineering approaches and is introduced using an illustrative example.

1 Modeling safety in engineering systems

Computational tools are instrumental to researching and developing new technologies. In order to represent an aspect of reality in a computer, this aspect must be modeled in a way that can be interpreted and worked with by a computer. Here is a list of common examples for computational tools in engineering systems:

Aspect/Domain	**Model**	**Associated scientific discipline**
Mechanical Design	CAD Model	Geometry
Aerodynamic analysis	Finite-Volume Model	Fluid Mechanics + Differential equations (Navier-Stokes equations)
Structural analysis	Finite-Element Model	Solid Mechanics + Differential equations (Navier-Lamé)
Thermal analysis	Finite-Element Model	Heat + Differential equations (Heat equation)
Electromagnetic analysis	Finite-difference Model	Maxwell equations

Table 1: Examples of modeling domains and their associated disciplines.

Modern society depends (by definition) on many safety-critical engineering systems (energy,transport, medical, etc.). Developing computational tools to investigate the safety of such systems is therefore essential.

Because safety is intimately linked to the perception of the individuals involved with the development of the system (clients, designers, users, etc.), it is difficult, if not impossible, to reduce safety to a single, measurable, physical quantity like heat, pressure, or electrical current. The existence of fundamental physical quantities is the foundation upon which are built most models of engineering systems since physics models (for example differential equations) can then be used to effectively simulate the system of interest.
The quantity that best approaches the properties required to model safety in engineering systems is *risk*.
Risk is the product of a loss of some form (life, financial, reputation, material, etc.), which depends on what designers whish to avoid, and the probability of occurrence of such a loss:

$$\text{Risk} = \text{Loss} \times \text{Probability of occurrence}$$

Although the nature of the loss is often easy to identify in many cases, the probability of occurrence given the complexity of real-world engineering systems makes this difficult. It is the probabilistic

aspect that makes reliability engineering central to modern safety analysis of engineering systems: reliability engineering is effectively the application of the field of probability and statistics to engineering problems.
[OK12] lists common and powerful techniques for calculating (or estimating) the reliability of complex engineering systems:

- Block-diagram analysis
- Fault tree analysis
- Markov Analysis
- Petri Nets

We present a rapid overview of each of these formalisms:

1.1 Block Diagram Analysis

Block diagram analysis in reliability engineering is the use of a reliability block diagram (RBD) to model how component reliabilities combine to determine overall system reliability and availability. The system is modeled as a set of blocks, each block corresponding to a component or subsystem that can succeed or fail. Blocks are connected according to the functional success logic: the path from input to output must be intact for the system to work.
Given failure rates, MTBF/MTTR, or reliability functions for each block, the following can be computed [NAS08]:

- System reliability $R(t)$ and unreliability as a function of time
- Availability and unavailability (with repair)
- Importance measures or criticality of components (which blocks most affect system performance)
- Effects of design changes (adding redundancy, changing maintenance strategy, etc.)

Block diagram analysis supports design trade-offs, verification of reliability requirements, and sensitivity analyses early and throughout the development lifecycle [NAS08].

1.2 Fault Tree Analysis (FTA)

Fault tree analysis was created in 1962 at Bell Telephone Laboratories by H.A. Watson for the U.S. Air Force Ballistics Systems Division to evaluate the safety of the Minuteman I ICBM launch control system.[Wat61, Wik26c, ICM99, VG69] Its first major applications were the safety evaluations of the Minuteman II missile system by Boeing and AVCO, and it was then adopted in commercial aerospace and nuclear power from the late 1960s onward.[Wik26c, ICM99, Liq12, Pet94]

1.3 State-space Analysis (Markov Analysis)

Markov state space analysis in reliability engineering is a modeling technique in which a system is represented by a set of discrete states (e.g. operational, degraded, failed, under repair) and the transitions between them, using a Markov process to compute reliability and availability over time.
The system is described by a state-space where each state encodes which components or modes are up or down, and transitions (with given failure and repair rates) satisfy the Markov property, i.e. the future depends only on the current state and not on the past history. By solving the balance equations of this Markov model, one obtains state probabilities as functions of time or at steady state, and system reliability or availability is calculated by summing the probabilities of all up states (with unavailability and risk measures from down or hazardous states) [NAS96].
Markov state space analysis is particularly useful for repairable and dynamic systems with standby redundancy, reconfiguration, or imperfect coverage, where static methods such as conventional reliability block diagrams or fault tree analysis are insufficient [Eur25].
Limitations of Markov State Analysis: need to assume that the probabilities or rates of all occurrences are constant and all events are statistically independent.

1.4 Petri Nets

Petri nets are a graphical and mathematical modeling formalism in which a system is represented by places, transitions and tokens to describe the flow of events and resources. In reliability and dependability engineering, (stochastic) Petri nets extend traditional methods such as reliability block diagrams and fault trees by capturing concurrency, resource sharing, repair, degraded modes and other dynamic behaviours that evolve over time [B+04, Vol10]. The marking of the net (distribution of tokens) defines the system state; by generating and analysing the reachability graph or using simulation, one can derive reliability and availability measures for complex, repairable and highly dynamic systems [B+04, Vol10].

2 A Unified Model type for reliability engineering?

The variety of the methods shown above begs the question: are there classes of models that are powerful enough to grasp all the aspects of the different modeling approaches? This section derives such a class of "super models" by building on the collection listed above.

1. **Explicit vs Implicit modeling of system states** The Markov analysis differs from the other formalisms in that it presents the states of the system *explicitly*, i.e. every state of the system is explicitly modeled ("failed", "working", "in repair", etc.). The strength of this formalism is the study of transitory behavior with little computation (i.e. a single deterministic calculation instead of a Monte-Carlo approach). The main weakness is for non-trivial systems, the number of states becomes intractable.
   Therefore, a formalism for practical reliability engineering requires an implicit representation of states: the system needs to be represented with variables that describe its configuration without needing to enumerate all the possible system states.

2. **Static vs Dynamic models** With the exception of the petri-nets, all the formalisms above are *static*, i.e. they do not capture changing systems or path dependent system (i.e. a sequence of events "A" then "B" is not the same as "B" then "A"). Path dependence is important for modelling engineering systems, particularly those with many human interfaces and software.

3. **Simple modeling of arbitrary logic** Despite being dynamic and implicit modeling of the state-space, Petri-nets tend to be difficult to use for arbitrary logic in engineering systems.

Let us look at what information would be required for a "unified" reliability engineering model: a set of state variables V would describe the configuration of the system. These variables need to be dynamic. Let us represent such a variable v in time (see Fig.1). In order for a state variable to change, or *transit* to another value, the values it can change to must be defined (i.e. a list of *transitions* needs to be defined). But a state variable cannot change value arbitrarily: there needs to be a set of conditions or *guards* that need to be met for the variable to transition. The requirement for arbitrary logic in the firing of the transition can be encoded in these conditions.
A model that contains this information is a **guarded transition system**.

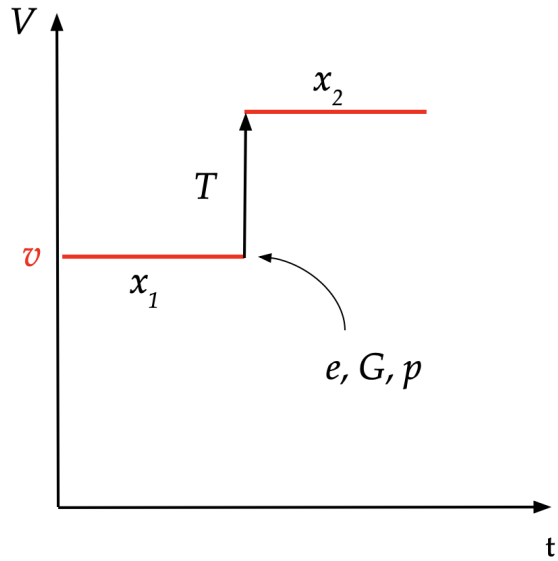

Figure 1: "x-t" diagram for a variable v, showing a transition with the associated event e, its associated set of guards G and p What is p?

A guarded transition system (in the sense of Rauzy et al. for reliability modeling) is defined as a 6-tuple $\langle V, E, T, \iota, H, B \rangle$ [Ant08]. More explicitly:

- V is a set of variables describing the system configuration.
- E is a set of symbols called events.
- T is a set of transitions, each transition being a triple $\langle G, e, P \rangle$ where G is a Boolean expression over V (the *guard* or pre-condition), $e \in E$ is an event, and P is an instruction over V (the post-condition).
- ι is an initial assignment of all variables in V.
- H and B are instructions over V called respectively the head and the body parts, used to compute the fix point after each transition firing.

A common informal way to phrase it: a guarded transition system is a state/event formalism where system states are encoded by variables and transitions are enabled only when their logical guard over those variables is true, upon which the associated event occurs and the post-condition updates the variables.
This collection of information permits the description of most engineering systems. However, the collection alone is insufficient to make the problem computationally tractable. For this, the information above must be translated into different mathematical models. This translation is called *flattening*. Are all GTS's flattened? What is the flattening process and what makes this process special from a computational perspective? What is the "fixed point" iteration? Why is this important?

3 Guarded Transition Systems by example

This section consists of building the Guarded Transition System formalism through an illustrative example. Some key concepts of reliability modeling of engineering systems are shown along the way.

3.1 The simple slot machine

Our illustrative example is that of a hypothetical slot machine. Our goal is to calculate our chances of winning. Our machine has N reels each having M different symbols. We will designate each set of symbols as a *state* of the machine. The total number of symbol combinations (and, therefore, the *state space*) is $\mathcal{S} = M^N$.
Let us represent all the possible states of a simple slot machine: only 2 reels ($N = 2$) and 3 symbols: "0","1","2" ($M = 3$). There are $\mathcal{S} = M^N = 3^2 = 9$ possible states for this machine. Suppose that each rotation of a reel corresponds to a *transition* from one state to the other; we can represent all the states of our simple slot machine in the diagram below:

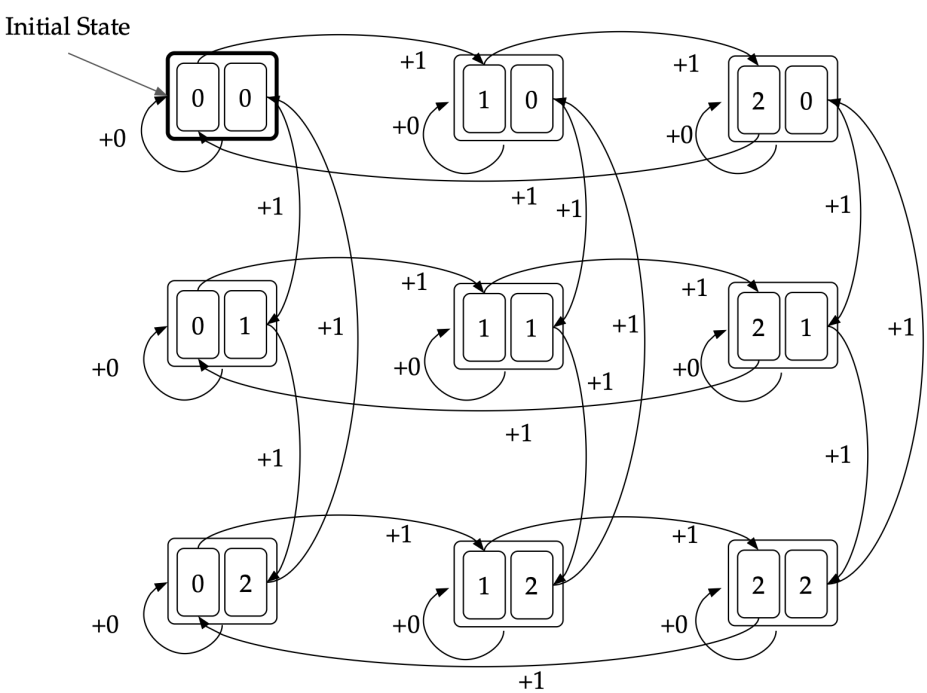

Figure 2: State transition diagram of the simple slot machine (2 reels, 3 symbols each). Note that this graph is only for $M = 3, N = 2$, and grows exponentially with M and N. Note that in this representation, there is no possibility for the reels to stay static. If there were, there would be transitions circling back to the state itself.

Observation

- State spaces are large, even for comparatively simple systems. Their size tends to grow exponentially with components and states for each component.
- If state spaces are large, methods requiring a full description of the state space (such as Markov chains) can only be used for exceedingly simple systems.
- For practical engineering systems, a mathematical formalism is required where an explicit description of the full state space is not necessary.

For the simple slot machine to be fair, each transition (arrows in Fig.2) should have an equal probability. If the reels were loaded, some transitions would have a higher probability than others.
Let us designate each reel of the machine as a component, for example R_1 and R_2 for a 2 reel slot machine. Since each reel can show a different number, let's say the number displayed by the reel is a state of the reel s_0, s_1, s_2 for our 3 numbers.
Therefore, the model of our simple slot machine has 2 components with 3 states each.

3.1.1 Playing the simple slot machine

Now we want to play the slot machine. Let's assume that one wins only if the symbols of both wheels are identical (i.e. the diagonal in Fig.2). If the slot machine is fair (all transitions have equal probability) and the number of spins is random, then the chance of winning is the number of states in the diagonal divided by the total number of states $P_{win} = \frac{M}{M^N} = M^{1-N}$. For the simple slot machine, $P_{win} = M^{1-N} = 3^{1-2} = 1/3$.
Lets see if we can derive this result with the formalisms discussed above:

Markov Analysis
If we were to rederive this result using a Markov state-space analysis, we would need to write the *stochastic transitional probability matrix* which has the form $P_{ij} = p_{s_i \to s_j}$ where $p_{s_i \to s_j}$ is the probability of transitioning from state s_i to state s_j. For the simple slot machine, we have:

$$P_{ij} = \frac{1}{3}\begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

We can calculate the probability of being in a given state after k timesteps with the equation: $p_i = P_{ij}^k I_i$ Where I_i is the initial state. According to Fig.2, $I_i = [1, 0, 0..., 0]$. For example, for $k = 3$, we have:

$$p_i = P_{ij}^3 I_i = \frac{1}{9}\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 2 \\ 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

As expected, the sum of the elements of p_i is 1. If we want to compute the *steady state* π_i of this slot machine, we use the fixed point equations to find the states: $\pi_i P = \pi_i$ Also, $\sum \pi_i = 1$. In this case, we have

$$\pi_i = \frac{1}{9}\begin{bmatrix}1\\1\\1\\1\\1\\1\\1\\1\\1\end{bmatrix}$$

To calculate the probability of "winning", we sum up the probabilites of the winning states, i.e. $P_{s_1} + P_{s_5} + P_{s_9} = 1/3$, which is the same result as above.
Finally, we can evaluate the stability of out simple slot machine by measuring the eigenvectors of P_{ij}. If all the eigenvalues are within the unit circle and the multiplicity of the eigenvector $e = 1$ is 1, then the system is *ergotic*, i.e. it converges to the steady state probability distribution shown above.

Petri Nets

Petri nets are a mathematical and graphical modeling language for describing systems where many events occur concurrently, asynchronously, or in a distributed way, representing states as "places" and events as "transitions" connected in a directed graph. They were introduced by the German computer scientist Carl Adam Petri in his 1962 doctoral thesis "Communication with Automata," building on ideas he first used as a teenager in 1939 to describe chemical processes, and have since been developed into a rich theory with many extensions.

Today, Petri nets are widely used to model, simulate, and verify discrete-event dynamic systems such as communication protocols, concurrent and distributed software, workflow and business processes, manufacturing and logistics systems, and control and reliability models, because they can naturally capture synchronization, parallelism, resource sharing, and conflicts in complex processes.

We model our simple slot machine using the petri-net shown in Fig.3:

1. There is a "source node" at the "top" of the net, emitting a new token at each beat.
2. the generated token can follow 3 transitions: it can increment the left wheel (equivalent to following a left to right arc in Fig.2), or the right wheel (equivalent to following a top to down arc in Fig.2 or get "flushed" down the sink transition on the right (equivalent to following a left to right arc in Fig.2). The fact that a token can go in either branch makes the system non-deterministic.
3. Once the "left" or "right Reel" places collect 3 tokens, the sink transitions activate and the places are empty again.

Counting the number of tokens in "left" and "right reel" reflects the state of our slot machine, see Fig.4.

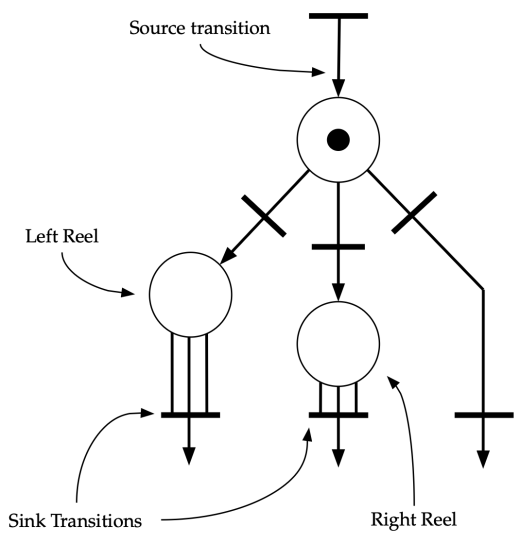

Figure 3: Petri net representation of the "simple" slot machine. At each time step, the source transition fires a token into the initial place (with always contains 1 token). This token has equal probability of going down the left transition (increment the left reel), middle transition (increment the right wheel) or incremental nothing at all (right transition).

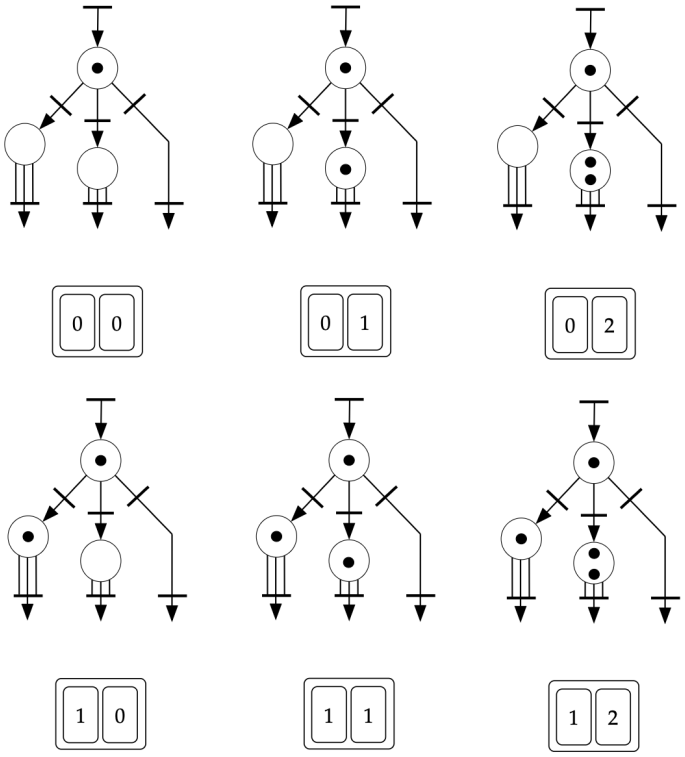

Figure 4: Illustration of different states (i.e. positions of the tokens) with the associated states below.

In order to calculate the probability of winning, the petri-net is used to simulate a large number of games. Every time the places "left reel" and "right reel" have the same number of token (i.e. the winning condition), a win is counted. The average number of wins converges to 1/3 as expected, see Fig.5.

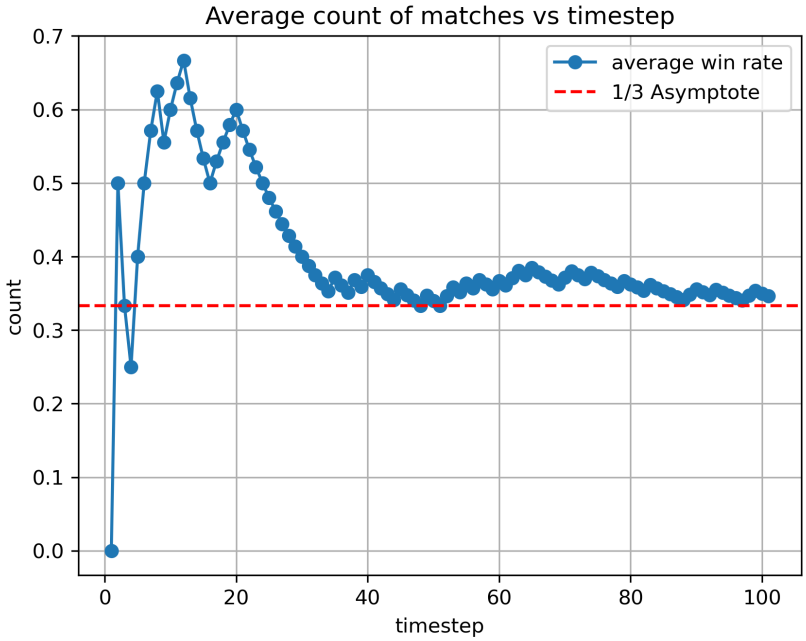

Figure 5: Convergence of the average win-rate of the slot machine using the petri-net simulation.

Guarded Transitions Systems One of the powerful properties of GTS' is that their mathematical properties permit allow them to be implemented using an object-oriented language (inheritance, composition, etc.). Alta-Rica is one such language.
To model our simple slot machine, we create a *class* "Reel", and model our slot machine as having two of these reels. To observe whether we have won or not, we need to implement an *observer* that counts the times that we win.

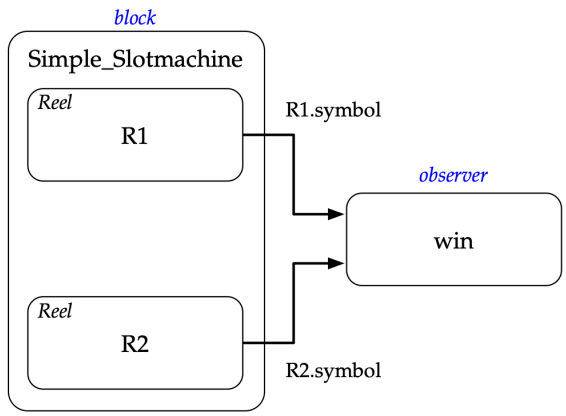

Figure 6: The "simple slot machine" as a guarded transition system. Note how simple it is: the system "block" is composed of 2 instantiations of the "Reel" class and a counter ("win") that counts the number of wins.

The following alta-rica model has a fundamental difference to the Markov and Petri implementations from above: it is continuous-time as opposed to discrete time. This means that we don't count the number of wins, but sum up the time spent in a winning state.
In order to model a reel, we use the knowledge:

- The probabilities of transitioning from one state to another do not change with time, i.e. the probability of survival follows an exponential function $S(t) = e^{-\lambda t}$.
- Both reels have the same properties $P_{turn,left} = P_{turn,right}$
- At any given instant, both reels have a probability of staying stationary of 1/3. Note that $P_{turn} = 1 - P_{no,turn}$

Therefore, we can calculate the turn rate $\lambda_{turn} = P_{turn}$ over a unit time t based on the following equations:

$$P_{no,turn,left} P_{no,turn,right} = P^2_{no,turn} = 1/3$$

Therefore, $P_{turn} = 1 - P_{no,turn} = 1 - \sqrt{1/3} = 0.422649731$. For N reels, we have

$$\lambda_{turn} = P_{turn} = 1 - P_{no,turn} = 1 - \sqrt{\frac{1}{N+1}}$$

```
class Reel
        Integer symbol (init = 0);

        event
                turn (delay = exponential(lambda));
                // there is a given probability that the reel turns at
                    timestep t.
                // The probability is constant and is always at 1-sqrt(1/3)
                // (P(turns) = 42.2649% ).

        parameter Real lambda = 0.422649731;

        transition
                turn : true -> symbol := mod (symbol + 1, 3);
 //             turn : true -> symbol := ((symbol + 1) mod 3);
                //post-condition: increment modulo 3

        // assertion

        // In this case, there is no need for an assertion since
        // the internal state of the Reel (R.symbol) is broadcast
        // and sufficient for the model.
end


block TwoReelsSystem

// instances of Reel
//component
  Reel R1;
  Reel R2;

// Observer: true when both reels show same symbol
// Boolean win (reset = false);

//assertion
//  win := (R1.symbol == R2.symbol);

observer Boolean win = (R1.symbol == R2.symbol);


end
```

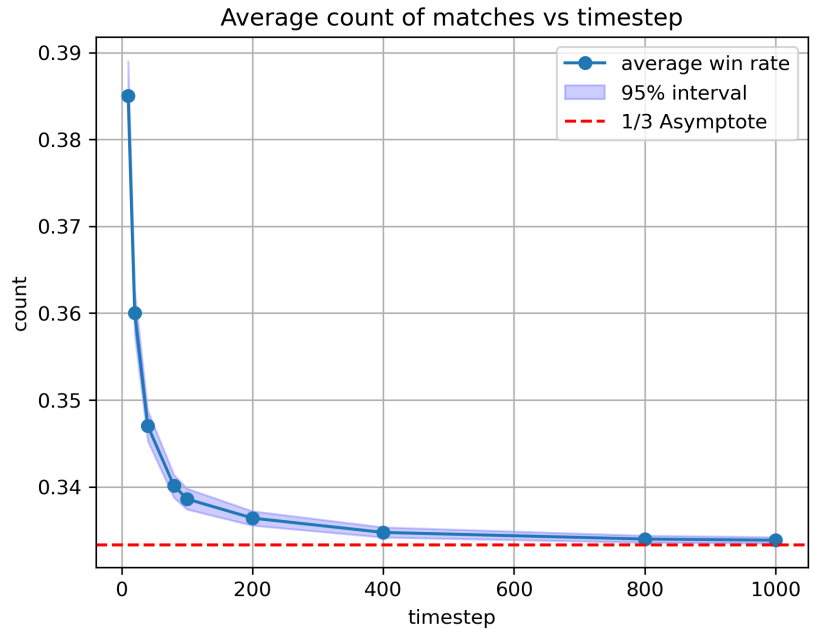

Figure 7: Convergence

3.2 Growing the simple slot machine

Let's see how our computational models cope with a increasing size of the slot machine. We can increase the number of symbols on each reel, but also the number of reels per machine. The chance of winning is still $P_{win} = M^{1-N}$.

Include the chord-diagrams of the different slot machines

Table 2: Comparison of Theoretical and Calculated Winning probabilities for 100 hours of operations with 1000 runs.

M	N	N_{states}	λ	P_{theory}	P_{calc}	Error (%)	T [h]	N_{runs}
3	3	27	0.3700	1/9	0.1183	6.47	100	1000
3	3	27	0.3700	1/9	0.1185	6.65	100	10000
3	3	27	0.37003947	1/9	0.1179	6.1	100	1000
3	3	27	0.3700	1/9	0.1122	1.26	1000	1000
10	3	1000	0.3700	1/100	0.0113174	13.17	1000	1000
10	3	1000	0.3700	1/100	0.0100168	0.168	10000	1000

3.3 The sticky slot machine

Let's assume that our slot machine is not as impartial as initially thought: it has not been cleaned in a long time, so of the reels stick to each other for certain combinations.

To be completed

4 Conclusions

References

[Ant08] Antoine Rauzy. Guarded transition systems for reliability studies. https://www.altarica-association.org/Documentation/pdf/Rauzy2008-GuardedTransitionSystemsForReliabilityStudies.pdf, 2008. Accessed 2026-02-22.

[B+04] Andrea Bobbio et al. Petri nets and dependability. In *Lecture Notes on Dependability*, pages 133–187. 2004. Preprint available from Università del Piemonte Orientale.

[Cav08] Cavac. Event tree analysis. https://cavac.at/cavacopedia/event%20tree%20analysis, 2008. Accessed 2026-02-22.

[Eur25] European Space Agency. Reliability prediction methods. https://handbook.reliability.space/, 2025. Digital handbook, accessed: 2026-03-06.

[ICM99] ICMA. The fault tree analysis (fta). http://www.icma.org.uk/06-8_fta.html, 1999. Accessed 2026-02-22.

[IfM15] University of Cambridge Institute for Manufacturing. Fmea (failure modes and effects analysis). https://www.ifm.eng.cam.ac.uk/research/dmg/tools-and-techniques/fmea-failure-modes-and-effects-analysis/, 2015. Accessed 2026-02-22.

[Liq12] LiquiSearch. Fault tree analysis - history. https://www.liquisearch.com/fault_tree_analysis/history, 2012. Accessed 2026-02-22.

[NAS96] NASA. Markov modeling for reliability and availability analysis. https://ntrs.nasa.gov/, 1996. Technical report, accessed: 2026-03-06.

[NAS08] NASA. System reliability assessment using block diagrams. https://extapps.ksc.nasa.gov/reliability/Documents/Preferred_Practices/1313.pdf, 2008. Accessed: 2026-03-06.

[OK12] Patrick D. T. O'Connor and Andre Kleyner. *Practical Reliability Engineering*. Wiley, Chichester, 5 edition, 2012.

[Pet94] R. H. Peters. Using fault tree analysis to focus mine safety research. Technical report, U.S. Bureau of Mines, 1994.

[R+75] N. C. Rasmussen et al. Reactor safety study: An assessment of accident risks in u.s. commercial nuclear power plants. Technical Report WASH-1400/NUREG-75/014, U.S. Nuclear Regulatory Commission, 1975.

[S+04] M. Stamatelatos et al. Pra history. Technical Report Reliability Engineering and System Safety lecture notes, U.S. Nuclear Regulatory Commission, 2004.

[SDH23] SDH. Failure mode and effects analysis: An overview. https://sdh.global/blog/business/failure-mode-and-effects-analysis-an-overview/, 2023. Accessed 2026-02-22.

[Sol23] Visure Solutions. History and evolution of fmea. https://visuresolutions.com/risk-management-fmea-guide/history-and-evolution/, 2023. Accessed 2026-02-22.

[Sta24] D. H. Stamatis. 75 years of fmeas: 1949–2024. *Quality Digest*, 2024.

[VG69] W. E. Vesely and F. F. Goldberg. Fault tree analysis of computer systems. In *Proceedings of the 1969 Fall Joint Computer Conference*, 1969.

[Vol10] Vlad Volovoi. Application of petri nets to reliability prediction of occupant safety systems with partial detection and repair. *Reliability Engineering & System Safety*, 95(6):606–613, 2010.

[Wat61] H. A. Watson. Systematic safety analysis of the minuteman launch control system. *Bell Telephone Laboratories (internal report)*, 1961.

[Wik26a] Wikipedia contributors. Event tree analysis — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Event_tree_analysis, 2026. Accessed 2026-02-22.

[Wik26b] Wikipedia contributors. Failure mode and effects analysis — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Failure_mode_and_effects_analysis, 2026. Accessed 2026-02-22.

[Wik26c] Wikipedia contributors. Fault tree analysis — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Fault_tree_analysis, 2026. Accessed 2026-02-22.

5 Appendix: History of System Safety Analysis

Safety analysis as a distinct engineering discipline emerged in the mid-20th century around high-hazard military and nuclear systems, with different techniques appearing in successive waves.

5.1 Fault tree analysis (FTA)

Fault tree analysis was created in 1962 at Bell Telephone Laboratories by H.A. Watson for the U.S. Air Force Ballistics Systems Division to evaluate the safety of the Minuteman I ICBM launch control system.[Wat61, Wik26c, ICM99, VG69] Its first major applications were the safety evaluations of the Minuteman II missile system by Boeing and AVCO, and it was then adopted in commercial aerospace and nuclear power from the late 1960s onward.[Wik26c, ICM99, Liq12, Pet94]

5.2 Failure mode and effects analysis (FMEA)

Failure mode and effects analysis originated in the U.S. military in the late 1940s as a structured method to evaluate reliability and prevent failures in munitions and mission-critical equipment.[IfM15, SDH23, Sol23, Wik26b] By the 1950s it was formalized in military standards, and in the 1960s it was adopted by aerospace organizations and NASA (e.g. Apollo) before spreading into automotive and general industrial product development from the 1970s onward.[IfM15, Sol23, Sta24]

5.3 Event tree analysis (ETA)

Event tree analysis in its modern form was developed in the late 1960s in UK Atomic Energy Authority design offices for reactor design and whole-plant risk assessment.[Wik26a, Cav08] The name "event tree" and systematic use of ETA were popularized in the 1970s by the WASH-1400 "Reactor Safety Study", which applied it to nuclear power plant safety.[Wik26a, S+04, R+75]

5.4 Probabilistic risk assessment (PRA)

The WASH-1400 Reactor Safety Study (1974–1975) integrated fault trees and event trees into a comprehensive probabilistic risk assessment framework for light-water reactors, quantifying accident sequences and their frequencies.[R+75, S+04, **?**] This work established PRA as a modern methodology for nuclear power plant safety and later influenced other high-hazard domains.[S+04, **?**]

Timeline and primary contexts Late 1940s–1950s: FMEA in U.S. military reliability engineering for munitions and mission-critical equipment.[IfM15, SDH23, Sol23]

Early 1960s: FTA at Bell Labs for Minuteman missile launch control safety, then missile and aircraft safety at Boeing and others.[Wat61, Wik26c, ICM99, Liq12]

Late 1960s: Forward-logic event tree methods for reactor and whole-plant safety at UK nuclear organizations.[Wik26a, Cav08]

Mid-1970s: WASH-1400 formalizes the combined use of FTA and ETA in a probabilistic risk assessment framework for nuclear power.[R+75, S+04, **?**]

Most of the early development is thus tightly coupled to military (munitions, missiles, aerospace) and civilian nuclear power programs, where high complexity and severe consequences motivated formal safety and reliability analysis methods.